

ZeBu[®] BSA Compliance Testing Reference Guide

Version W-2025.06, September 2025

SYNOPSYS[®]

Copyright and Proprietary Information Notice

© 2025 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

About This Book	5
Intended Audience	5
Contents of This Book	5
Related Documentation	6
Typographical Conventions	6
Synopsys Statement on Inclusivity and Diversity	7

1. Introduction to BSA Compliance Testing	8
Overview of the Solution	8
Purpose of the Solution	9
Features and Benefits	10
Product Flow	10

2. Installation and Setup	12
--	-----------

3. Integrating the BSA Compliance Tests	13
--	-----------

4. BSA Compliance Test Function	15
NIC Emulator	15
Transmit and Receive Buffer Descriptors	16
Memory Mapped Registers	16
Function Memory Buffer	22
Distribution Functions	23
Sideband Commands	24
Sideband Command Buffer	27
Sideband Command Control Registers	29
Sideband Command Set ID	29
Sideband Commands For Controlling Response and Requests	30
Ping Sideband Command Processor	31
Completion Status Override (UR/CRS/CA)	31
Hold Response to Create Timeout on Root Port	32

Contents

MSI/X Interrupt	32
Continuous MSI Interrupts	32
Upstream Read	33
Upstream Random Read	34
Upstream Write	35
Upstream Random Write	36
Random Upstream Atomic Operation	37
Request With Different AT Bits	38
Error/PME/INTx Message	38
Send Data With Poison Bit Set	40
Set PASID	40
Set No Snoop	40
Set Traffic Class	41
Set Ordering	41
Set TLP Processing Hint	41
Start Recorder	41
Cancel	43
Independent Sideband Access	43
Setting the Priority of Upstream Read Requests Over Write Requests	45
ACS Testing	47
Address Invalidation Testing	47

5. Appendix: Supported Software Drivers	49
Sideband Channel Software Driver	49
Hardware Abstraction Layer	49

Preface

This chapter has the following sections:

- [About This Book](#)
- [Intended Audience](#)
- [Contents of This Book](#)
- [Related Documentation](#)
- [Typographical Conventions](#)
- [Synopsys Statement on Inclusivity and Diversity](#)

About This Book

The *ZeBu® BSA Compliance Testing Reference Guide* describes pre-silicon ARM BSA compliance testing, its benefits, and usage model using the BSA add-on for the *Zebu Virtual Device Solution*. The document uses a PCIe subsystem as an example. For details on installation, configuration, and usage of the ZeBu Virtual Device Solution, refer to the *ZeBu® Virtual Device Solution User Guide*.

Intended Audience

The engineers should have basic to advanced knowledge of using the following products: PCI Express, Simulation Acceleration, ZeBu platform.

Contents of This Book

The *ZeBu® BSA Compliance Testing Reference Guide* has the following chapters:

Chapter	Describes...
Introduction to BSA Compliance Testing	Provides an overview, purpose, and features and benefits of the solution.
Installation and Setup	Describes the prerequisites and the licensing requirements for the compliance tests.
Integrating the BSA Compliance Tests	Explains the modules required to compile and run the BSA compliance tests.

Chapter	Describes...
BSA Compliance Test Function	Describes in detail the BSA compliance test function.
Appendix: Supported Software Drivers	Describes the software driver components of VDM that ease the implementation and execution of BSA compliance tests.

Related Documentation

For details on installation, configuration, and usage of the BSA compliance tests, refer to the *ZeBu Virtual Device Model Solution User Guide*.

Typographical Conventions

This document uses the following typographical conventions:

To indicate	Convention Used
Program code	<i>OUT</i> <= IN;
Object names	<i>OUT</i>
Variables representing objects names	<sig-name>
Message	Active low signal name '<sig-name>' must end with _X.
Message location	<i>OUT</i> <= IN;
Reworked example with message removed	<i>OUT_X</i> <= IN;
Important Information	NOTE: This rule...

The following table describes the syntax used in this document:

Syntax	Description
[] (Square brackets)	An optional entry
{ } (Curly braces)	An entry that can be specified once or multiple times

Syntax	Description
(Vertical bar)	A list of choices out of which you can choose one
... (Horizontal ellipsis)	Other options that you can specify

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction to BSA Compliance Testing

Arm Base System Architecture (BSA) is a hardware design and firmware standard for Arm-based System-on-Chip (SoC). The primary goal of formulating the BSA standard is to create a system architecture so that a single suitably built operating system (OS) image can run on all compliant hardware. The following diagram shows the software stack in BSA compliant SoCs:

Figure 1 Software Stack in BSA Compliant SOCs



See the following sections:

- [Overview of the Solution](#)
- [Purpose of the Solution](#)
- [Features and Benefits](#)
- [Product Flow](#)

Overview of the Solution

The BSA Compliance Testing solution is developed by Synopsys in collaboration with Arm using Arm reference designs and elemental kits. It includes Arm's ACS test suites, C-models for system realization, bring-up assistance, and performance validation capabilities.

The solution also measures key performance metrics, such as channel throughput, Rd/Wr/ Interrupt latency, and Bandwidth utilization. The solution is built to natively support ZeBu Emulation and VCS simulation.

Purpose of the Solution

Before proceeding with the purpose of the solution, you need to understand the reasons why pre-silicon compliance testing must be performed. Refer to [Table 1](#) for more details.

Table 1 Reasons for Performing Pre-Silicon Compliance Testing

Defects commonly seen during bring up ...	With a significant impact ...	And caused by a non-compliant hardware
• OS fails to start or stops responding • Invalid PCIe hierarchy, no Root Port discovered at all • Incorrect ECAM enumeration and devices not discovered • Endpoint interoperability issues • CPU writes with certain byte enables does not work, data corruption and software errors	• Windows Server and Client do not work with non-compliant PCIe ECAM • Linux maintainers not willing to accept patches for PCIe compliance • No firmware workaround exists for systems with extensive PCIe hierarchies • Key features such as virtualization and RAS are not available with existing workarounds • Security updates are limited for custom OS distributions	• Non-standard PCIe ECAM, or no ECAM at all • No PCIe 64 bit MMIO • PCIe byte access enable • PCIe ghost devices • Limited PCIe DMA for USB XHCI • Non standard GIC • Non standard UART Timer issues
• Mitigation in software is costly and challenging, even if possible	• Uncompetitive or not acceptable at all	• Identifiable and preventable early in design cycle

So, pre-silicon compliance testing helps:

- Reduce SoC developer risks by catching BSA compliance bugs/defects before tape-out.
- Ensures that off-the-shelf operating systems can boot and function without any custom patches
- Increases market size and improve product attractiveness

Features and Benefits

The Synopsys Arm BSA Compliance Testing Solution helps achieve the following for the Verification Platform:

- Usable in various functional platforms, such as simulation only, simulation acceleration, and emulation
- Uses standard compliant design interfaces, such as SystemVerilog HDL (Synthesizable), C-based transactors

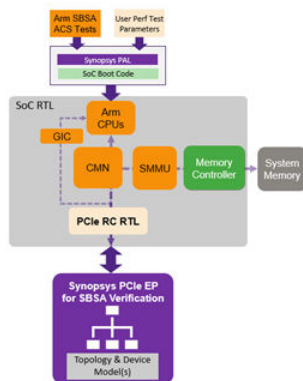
Arm BSA Compliance Testing offers the following benefits:

1. Enable high coverage of Base System Architecture (BSA) compliance pre-silicon verification: PCIe subsystem
2. Provides a generic device-side target for verification of PCIe links
3. Supports arbitrary device topologies

Product Flow

A high-level solution for running Arm BSA compliance test suite on a PCIe subsystem is shown in [Figure 2](#).

Figure 2 BSA Compliance Test Suite Solution for PCIe EP Subsystem Verification



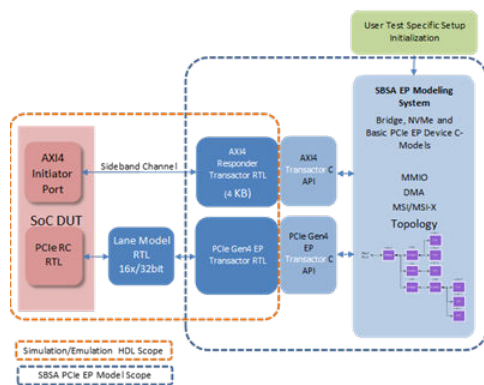
In this solution, the SoC block shows the Arm CPU and other IPs of a system under compliance testing. The firmware block consists of a bare metal driver named as `pcie_sideband_driver` and this driver is used by the PAL layer to make use of sideband commands exposed by the SBSA EP model(s).

Finally, the block at the bottom depicts the Synopsys BSA Virtual Device Model (VDM) solution that implements the Arm BSA compliance test suite for PCIe subsystems. [Figure 3](#) shows the topology of the solution.

Note:

It is up to you to define a suitable topology to execute the objectives of the individual test. In addition, you need to ensure using a supported version of PCIe or CXL transactors. For details about supported versions, refer to the ZeBu Virtual System Adaptor release notes.

Figure 3 Example Test Topology for PCIe Subsystem Arm BSA Compliance Verification



2

Installation and Setup

The BSA compliance testing solution is an add-on to VDM. For details about installing and setting up BSA and VDM, refer to the *ZeBu® Virtual System Adaptor Installation Guide*.

3

Integrating the BSA Compliance Tests

You need the following modules to compile and run the BSA compliance tests:

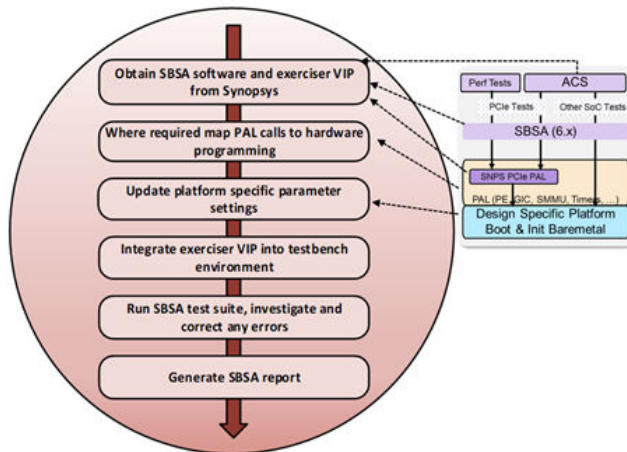
1. The `PAL bare metal exerciser` module, it would be part of the Synopsys package and the user needs to compile ACS tests suite using this module.
2. The `pci_sideband_driver` module, it would be part of Synopsys package and the user needs to use the static library, that is, `libpcie_sideband_driver.a` to link the ACS tests suite.
3. The `log` method, a method named as `c_print` needs to be implemented by the user, and this method is declared in the `pcie_sideband_driver/inc/log.h` file.

A sample implementation of `c_print` method:

```
#include <stdio.h>
#include <stdarg.h>
/**
 * @brief Wrapper provides printf functionality around pal_print().
 *        The Synopsys pcie_sideband_driver requires this
 * @param format An ascii string with formatting
 * @param ... data to be formatted
 */
int c_print(const char *format, ...)
{
    char print_buffer[4096];
    va_list args;
    va_start(args, format);
    int count = vsnprintf(print_buffer, sizeof(print_buffer), format,
args);
    if (count >= sizeof(print_buffer))
    {
        pal_print("WARNING, next string was too long to be formatted.",
0);
    }
    va_end(args);
    pal_print(print_buffer, 0);
    return count;
}
```

The steps to run the BSA compliance test are described in the diagram below:

Figure 4 Integrating the BSA Test



The BSA suite can be used to ensure portability of an OS image between hardware platforms compliant with this ACS.

Arm ACS tests along with non-platform specific API drivers are available from Synopsys.

Some Arm specific API drivers are also delivered as reference.

To check all rules of the BSA, the ACS suite must be run in a pre-silicon environment.

It is recommended the partner fold the BSA suite into a SW library to aid portability between testing environments.

4

BSA Compliance Test Function

For details about configuring the PCIe link and device topology used to execute the BSA compliance tests, refer to ZeBu Virtual Devices Model Solution User Guide.

Define a suitable topology to execute the objectives of the individual test.

The following BSA-specific components and configuration options are available in addition to the standard PCIe components and configuration options.

See the following sections in this chapter:

- [NIC Emulator](#)
- [Distribution Functions](#)
- [Sideband Commands](#)
- [Independent Sideband Access](#)
- [Setting the Priority of Upstream Read Requests Over Write Requests](#)

The pre-defined BSA Tester Function can be used to execute ARM BSA compliance tests. It includes compliance-related commands and response capabilities to trigger specific upstream traffic, modify the normal responses of the test function, or inject errors. Multiple instances of the BSA Test Function can be present and concurrently active in a PCIe Topology, to create the necessary traffic patterns.

The BSA test function is an extension of the standard PCIe function provided by VDM. The following predefined additional features are available to implement and execute BSA compliance tests.

NIC Emulator

The BSA Test Function includes an Input-Output DMA transfer queues that are able to mimic the operations of a NIC card and create various traffic profiles. The programming model is similar to that of an Intel Ethernet controller, through the memory-mapped registers in the BAR 0/1 region and transmit or receive descriptor ring buffers. For more details, see the PCIe GbE Controllers Open Source Software Developer's Manual (<https://www.intel.com/content/dam/www/public/us/en/documents/manuals/pcie-gbe-controllers-open-source-manual.pdf>)

Transmit and Receive Buffer Descriptors

The transmit and receive buffer descriptors use the same format as described in the following table. In a transmit descriptor, the 'Length' field specifies the number of bytes to transmit from the specified address. In a receive buffer, the 'Length' field indicates the maximum number of bytes that can be received at the specified address. The 'D' bit is set by the Tester Function when the content of the buffer has been sent or received. For receive buffers, the 'Length' field is updated with the actual number of bytes that were received. If the 'O' bit is set by the Tester Function, it indicates that the amount of byte received overflowed the available buffer and the updated 'Length' field has a value larger than the original value. Reserved bits should be cleared.

Newly available Transmit Buffer Descriptors are fetched by the Tester Function based on the new value of the TXD.TAIL register. All TxBDs between the previous TAIL and the new one, up to $2^{\text{IOQCTL.PFSIZ}}$, are fetched using a single DMA transfer (unless a 4K boundary is crossed or the FIFO is wrapped back to a lower offset, in which case two DMA transfers are necessary). The 'D' bit is updated by rewriting the entire TxBD content for up to $2^{\text{IOQICR.BATCH}}$ consecutive TxBDs in a single DMA transfer, whenever a Tx interrupt is issued (whether masked or not), or the TXD.HEAD wraps around to a lower offset.

Newly available Receive Buffer Descriptors are fetched by the Tester Function based on the new value of the RXD.TAIL register. All RxBDs between the previous TAIL and the new one, up to a maximum of $2^{\text{IOQCTL.PFSIZ}}$, are fetched by the Tester Function using a single DMA transfer (unless a 4k boundary is crossed or the FIFO is wrapped back to a lower offset, in which case two DMA transfers are necessary). The fields of a RxBD are updated by rewriting the entire RxBD content for up to $2^{\text{IOQICR.BATCH}}$ consecutive RxBDs in a single DMA transfer, whenever a Rx interrupt is issued (whether masked or not), or the RXD.HEAD wraps around to a lower offset.

Figure 5 Transmit / Receive Buffer Format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Buffer Address																																00h
Length (in bytes)																																04h
Reserved																																08h
																														O	D	0Ch

Memory Mapped Registers

The following memory-mapped registers in the BAR0/1 address space are used to configure the NIC Emulation function of the Tester Function. All memory-mapped registers must be accessed using aligned DWORD or QWORD accesses.

CTRL: Function Control Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
BDSZ				NQ												Rsvd												RXE	TXE	00h		

Field	Type	Description	Initial Value
TXE	RW	Transmit Enable	0
RXE	RW	Receive Enable	0
NQ	RW	Number of requested and active I/O queues. The value saturates at the maximum number of queues supported.	000h
BD SIZ	RW	Buffer Descriptor size in bytes, specified as a power-of-two. Minimum value is 4 (or 16 bytes). Maximum is 12 (or 4096 bytes).	4

ISR: Interrupt Source Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
BDSZ				NQ												Rsvd												RXE	TXE	00h		

Field	Type	Description	Initial Value
TX	RC	An I/O Queue has a Tx interrupt request.	0
RX	RC	An I/O Queue has a Rx interrupt request.	0
RG	RC	An I/O Queue has a Rg interrupt request.	0

ICR: Interrupt Control Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
																													RG	RX	TX	08h

Field	Type	Description	Initial Value
TX	RW	Mask (prevent) all Tx interrupts, if set. Will still set the TX bit in the ISR register.	0
RX	RW	Mask (prevent) all Rx interrupts, if set. Will still set the RX bit in the ISR register.	0
RG	RW	Mask (prevent) all Rg interrupts, if set. Will still set the Rg bit in the ISR register.	0

MSIX : MSI-X Table

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Msg Addr																																IV << 4 + 10000h
																																IV << 4 + 10004h
																																IV << 4 + 10008h
Msg Data																																IV << 4 + 1000Ch
Vector Control																																IV << 4 + 1000Ch

2048 MSI-X interrupt vectors are supported.

Reserved

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Reserved																																18000h
																																..
																																1FFFCCh

2048 MSI-X interrupt vectors are supported.

Reserved

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Reserved																																18000h
																																..
																																1FFFCCh

IOQCTL[#]: I/O Queue Control Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
FIFOLEN																Rsv d	PFSZ				Reserved								RXE	TXE	Q# << 6 + 20000h	

Field	T ype	Description	Initial Value
TXE	RW	Enable frame transmission. The Tx FIFO is reset whenever this bit is cleared.	0
RXE	RW	Enable frame reception. The Rx FIFO is reset whenever this bit is cleared.	0
PFSIZ	RW	Maximum Buffer Descriptors to prefetch in a single request, specified as 2^PFSZ.	0x7
FIFO LEN	RW	Number of descriptors in the Tx and Rx ring buffers	0

IOQISR[#]: I/O Queue Interrupt Status Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Reserved																												Rg	Rx	Tx	Q# << 6 + 20004h	

Field	Type	Description	Initial Value
Tx	RC	"2^BATCH" number of frames have been transmitted, or the Tx FIFO became empty.	0
Rx	RC	"2^BATCH" number of frames have been received, or the Rx FIFO became empty.	0
Rg	RC	The Receive Frame Generator is done.	0

IOQICR[#]: I/O Queue Interrupt Control Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Rsv	BATCH				IV											INTERVAL								Rsv	Rg	Rx	Tx	Q# << 6 + 20008h				

Field	Type	Description	Initial Value
Tx	RW	If set, prevent the Tx FIFO from generating an interrupt request (but the Tx bit in the Interrupt Status Register is still set).	0
Rx	RW	If set, prevent the Rx FIFO from generating an interrupt request (but the Rx bit in the Interrupt Status Register is still set).	0
Rg	RW	If set, prevent the Receive Frame Generator from generating an interrupt request (but the Rg bit in the Interrupt Status Register is still set).	0
INTERVAL(a)	RW	Minimum interval, in units of 1000 PCIe clock cycles, between interrupt requests using the specified vector. If different queues use the same interrupt vector, the interval applies to the last time an interrupt was requested by that queue, not the other queues using the same vector. A value of 0 indicates no minimum interval and interrupts are requested immediately.	00h
IV	RW	Generate an interrupt request using the specified MSI-X vector.	00h

BA RW Generate a Tx (Rx) interrupt and update sent (received) Buffer Descriptors whenever 0h
TCH 2^BATCH number of frames has been sent (received) or when the Tx (Rx) FIFO is empty (full).

(a) The INTERVAL only works in steady state when interrupts are continuously coming and eventually elapsing the specified minimum time interval. If transfers stop, and thus interrupts stop coming in, pending interrupts are not signaled. Pending interrupts can be flushed by injecting BATCH frames after the specified time interval has already elapsed.

IOQPAS[#]: I/O Queue PASID Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
V	Reserved							P	E	Rsvd	PASID																				Q# << 6 + 2000Ch	

Field	Type	Description	Initial Value
PA SID	RW	Process Address Space ID	00000h
E	RW	Execute Permission Requested	0
P	RW	Privileged Mode Requested	0
V	RW	Prefix is valid for this queue and is prepended to all upstream requests if PASID is enabled in the function's PASID capability.	0

TXDBA[#]: Transmit Descriptor Base Address

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Base Address																																Q# << 6 + 20014h
Base Address																Reserved																Q# << 6 + 20018h

Field	Type	Description	Initial Value
Base Address	RW	Base address of the Transmit Buffer Descriptor ring. Must be 4k aligned.	N/A

TXD[#]: Transmit Descriptor FIFO Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
HEAD																TAIL																Q# << 6 + 2001Ch

Field	Type	Description	Initial Value
TAIL	RW	Start of SW-owned Transmit Descriptors. Resets to 0x0000 whenever a '0' is written to 'IOQCTL.TXE'	0
HEAD	R	Start of Test Function-owned Transmit Descriptors. Resets to 0x0000 whenever a '0' is written to 'IOQCTL.TXE'.	0

Data is transmitted as soon as possible whenever the queue and transmission is enabled, and a transmit buffer descriptor is available to be transmitted. Transmitted data is sunk and ignored. The FIFO is empty whenever TAIL == HEAD.

RXDBA[#]: Receive Descriptor Base Address

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Base Address																																Q# << 6 + 20020h
Base Address																Reserved																Q# << 6 + 20020h

Field	Type	Description	Initial Value
Base Address	RW	Base address of the Receive Buffer Descriptor ring. Must be 4k-aligned.	N/A

RXD[#]: Receive Descriptor FIFO Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
HEAD																TAIL																Q# << 6 + 20028h

Field	Type	Description	Initial Value
TAIL	RW	Start of SW-owned Receive Descriptors. Resets to 0x0000 whenever a '0' is written to 'IOQCTL.RXE'.	0

H R Start of Test Function-owned Receive Descriptors. Resets to 0x0000 whenever a '0' 0
EAD is written to 'IOQCTL.RXE'.

RXGCTL[#]: Receive Generator Control Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
LENGTH																COUNT																Q# << 6 + 2002Ch

Field	T	Description	Initial Value
COUNT	RW	Number of receive packets to generate	0
LENGTH	RW	Length of generated receive packets	0

Data is “received” whenever the queue and reception are enabled, and the ‘COUNT’ and ‘LENGTH’ fields are non-zero. Receive data is filled with the following repeating 64-bit pattern.

63	48	47	32	31	16	15	0
IOQ #		RXGCTL.COUNT-i		0hBEEF		QWORD#	

Reserved

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Reserved																Reserved																Q# << 6 + 20030h
Reserved																Reserved																Q# << 6 + 20034h
Reserved																Reserved																Q# << 6 + 20038h
Reserved																Reserved																Q# << 6 + 2003Ch

Function Memory Buffer

The remaining address space in the BAR0/1 resource after offset 0x40000 is available as local memory buffer for in-function transfers instead of DMA, or for general purpose I/O testing. The local memory buffer address space extends to the end of the BAR0/1 resource and defaults to 0x000FFFFF. It can be increased if necessary using the <virtDev:SizeLog2> XML element. It specifies the total size, as a power-of-2, of the

BAR0/1 resource - not just the size of the scratch memory buffer. Minimum value is 18. Maximum value is 63. Default is 20.

A sparse memory model is used to model the local memory buffer. By default, the sparse memory line size is 256 bytes, but it can be configured to a larger or smaller value using the `<virtDev:LineSizeLog2>` XML element. It specifies the total size, as a power-of-2, of the sparse memory line used to model the local memory buffer located at the end of the BAR 0/1 region. Minimum value is 2. Maximum value is `<virtDev::SizeLog2>`. Default is 8 (256 bytes).

By default, the BSA Tester function responds as fast as possible to all memory requests (unless a Sideband HOLD command is in effect). The response time of local memory buffer read accesses can be increased to a specified minimum number of PCIe clock cycles to model various device latencies using the `<virtDev:RespDelay>` XML element. Unlike the "Hold Response" sideband command, this delay configuration is permanent and applies to all responses from the local memory buffer.

The following example shows the optional XML configuration element for a BSA Tester Function instance specifying the default values.

```
<virtDev:Function>
  <virtDev:Name>BSA</virtDev:Name>
  <virtDev:BAR0>
    <virtDev:SizeLog2>20</virtDev:SizeLog2>
    <virtDev:LineSizeLog2>8</virtDev:LineSizeLog2>
    <virtDev:RespDelay>0</virtDev:RespDelay>
  </virtDev:BAR0>
</virtDev:Function>
```

Distribution Functions

The Upstream Read and Upstream Write commands may use a random distribution function to determine the length of the next request. There are eight available distribution function slots. Each BSA Tester Function may have a different set, except for function #0 which is hard-coded to the "NULL" function.

Distribution functions are functions that take a single 16-bit unsigned value (DWLEN) and return the number of DWORD to be used in the next memory request, as an unsigned 16-bit number. The returned value must be between 1 and DWLEN inclusively. For example, the predefined "NULL" distribution function hard-coded in slot 0 of all Tester Functions is defined as follows.

```
uint16_t nullFct(uint16_t DWLEN, void* &state)
{
    return DWLEN;
}
```

If the specified distribution function slot is not implemented, the "NULL" function is used.

User-defined distribution function implementations are assigned to specified slots in each individual Tester Function using the

`SNPS::VirtDev::BSA::PCIEFunction::Tester::defineDistribution()` method. For example, the implementation of a 80/20 bi-modal distribution in slot #1 of a Tester Function as following.

```
uint16_t paretoFct(uint16_t DWLEN, void* &state)
{
    const uint16_t cMinSize = 64/4;
    const uint16_t cMaxSize = 1536/4;

    if (rand() % 100 < 80) return cMinSize;
    if (DWLEN < cMaxSize) return DWLEN;
    return cMaxSize;
}

SNPS::VirtDev::DeviceEnv::Env myEnv(...);
auto fn = myEnv.getFunction(1, 0, 0);
fn->defineDistribution(1, paretoFct);
```

An optional opaque pointer, initially set to NULL for each distribution function instance in each BSA Tester Function, can be used to maintain state information across invocations. The following example shows how a random-stable distribution function could be implemented.

```
uint16_t stableFct(uint16_t DWLEN, void* &state)
{
    if (state == NULL) state = new std::default_random_engine;
    auto rng = (std::default_random_engine*) state;

    std::uniform_int_distribution<uint16_t> dist(64/4, DWLEN);
    return dist(rng);
}
```

Sideband Commands

The BSA Tester Function can execute various commands to modify its responses, or to initiate upstream traffic with various patterns. These commands are submitted by writing to command buffers located in a memory-mapped BAR region in the BSA Tester Function. Those command buffers may be accessed via the PCIe interface or via the sideband channel.

The presence and configuration details of the sideband command buffers is advertised through a Designated Vendor-Specific Extended Capability Descriptor.

Sideband Extended Capability

Bit Range	Field Name	Value	Offset
31-20	Next Capability Offset	0014h	00h
19-16	1h	16c3h	04h
15-8	0023h	5B5Ah	08h
7-4	0014h		
3-0	16c3h		
31-28	SCSI		
27-24	SCNUM		
23-20	SCSIZ		
19-16	SDSIZ		
15-8	5B5Ah		
7-0	SC Offset		
31-28	SCBIR		0Ch
27-0	SD Offset		
31-28	SDBIR		10h

Name	Mode	Reset Value	Description
SCSI	RO	Impl.Specific	Sideband command set id.
SCSIZ	RO	3	Number of DWORDS in a single sideband command buffer, specified as 2^SCSIZ.
SCNUM	RO	Impl. specific	Total number of sideband command buffer available, not including the sideband command control registers.
SDSIZ	RO	Impl. specific	Number of bytes available in the sideband data region, specified as 2^SDSIZ. A value of 0 indicates that no sideband data buffer is available.
SCBIR	RO	Impl. specific	Indicates which one of the function's Base Address Registers located beginning at address 10h in the function's configuration address space is used to map the sideband commands into memory space.
SC Offset	RO	Impl. specific	Offset within the memory region where the first sideband command descriptor resides. The lower 3 bits (containing SCBIR) are masked to produce a QWORD-aligned address.
SDBIR	RO	Impl. specific	Indicates which one of the function's Base Address Registers located beginning at address 10h in the function's configuration address space is used to map the sideband data into memory space.
SD Offset	RO	Impl. specific	Offset within the memory region where the sideband data location starts. The lower 3 bits (containing SDBIR) are masked to produce a QWORD-aligned address.

ZeBu® BSA Compliance Testing Reference Guide
W-2025.06

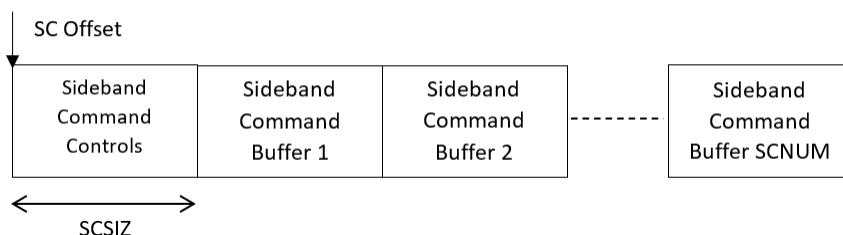
by using the SC Offset, SCBIR, SD Offset and SDBIR fields in the Sideband Extended Capability descriptor, similar to that used for MSI-X.

Sideband commands are structured as command descriptions located in a memory-mapped command buffer. The minimum size of the memory resource is TBD. An optional sideband memory-mapped data buffer may also be used if commands require more data than is available in the pre-defined command descriptor. Command descriptors refer to data in the sideband data memory using an offset relative to the first sideband data buffer location.

These two sideband memory resources must be mapped to a non-prefetchable 32-bit address region by the device enumeration process. The sideband command and data buffers may be located in the same contiguous memory space, but at different non-overlapping offsets.

Sideband commands are specified using SCSIZ-byte descriptors, located in contiguous memory locations in the memory-mapped region indicated by the SCBIR and SC Offset values. The first sideband command buffer is reserved and contains the sideband command control registers.

Figure 7 Sideband Extended Capability



The following subtopics are covered in this section:

- [Sideband Command Buffer](#)
- [Sideband Command Control Registers](#)
- [Sideband Command Set ID](#)
- [Sideband Commands For Controlling Response and Requests](#)

Sideband Command Buffer

Every sideband command buffer (except for the first one) has the following format:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
WEIGHT				CID												OPC				STATUS				Rev	C	N	V	00h				
DWORD[0]																																04h
...																																...
DWORD[SCSIZE-2]																																SCSIZ-1
																																SCSIZ

Name	Mode	Reset Value	Description
V	RW	0	Command is valid. Set by the host to indicate that the buffer contains a valid command. Cleared by the host to indicate that the completion of the command has been acknowledged.
N	RO	0	Command is new. Set whenever the host writes a '1' to the V bit. Cleared by the device to acknowledge the new command and indicate that it is being processed.
C	RO	0	Command is completed. Set by the device when the command has been completed and indicates the buffer contains a valid response. Cleared by the device at the same time it clears the N bit.
STATUS	RO	0	Command execution status code 0x0 Command has not yet completed 0x1 Command completed successfully 0x2 Command was canceled by user 0x3..0x7 Command-specific completion status 0xE Error in command arguments 0xF OPC is unknown or unsupported Others reserved
OPC	RW	n/a	Command opcode.
CID	RW	n/a	Command identifier. CID should be unique but need not be. CID 0x0000 and 0xFFFF are reserved.
WEIGHT	RW	n/a	Relative weight of the command
DWORD[i]	RW	n/a	Command-specific arguments and response. See the description of individual comments.

Command Weight

This parameter is only applicable to commands that execute more than once before completing or being explicitly canceled, such as the “Random Upstream Read” command. It is ignored (and should be 0) in all other commands. When multiple repeating commands can be executed, the command selection arbiter uses an interleaved weighted round-robin algorithm to select the next command to execute, where the weight of each command is specified as 2^{WEIGHT} . A command with a higher weight (maximum is 16) will execute more often than a command with a lower weight (minimum is 1).

Command Execution State

The following table shows the sequence of C/N/V bits during the execution of a sideband command.

Table 3 Sequence of C/N/V Bits When Executing Sideband Commands

C	N	V	Changed by	Description
x	0	0	N.A.	Command buffer is empty
x	0	0 -> 1	Host	A new command has been submitted by the host (causes N 0->1)
x	0 -> 1	1	Device	A new valid command has been indicated by the host
x	1	1	N.A.	A new command is in the buffer but has not yet been acknowledged by the device because the host has not written to NEWCMDS yet.
x -> 0	1 -> 0	1	Device	The command has been acknowledged by the device and is being executed.
0 -> 1	0	1	Device	The command is completed, and the buffer contains a valid response
1	0	1 -> 1	Host	Host has acknowledged the command completion and the buffer contains a new command (causes N 0->1)
1	0	1-> 0	Host	Host has acknowledged the command completion and the buffer is invalid
0	x	1 -> x	Host	Illegal
x	1	1 -> x	Host	Illegal

Sideband Command Control Registers

The sideband command control registers are located in the sideband command buffer #0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
Reserved																IVECT						I	Rsvd				0h				00h	
NEWCMDS								ACKCMDS								CPLCMDS																04
																																...
Reserved																																SCSIZ-4
																																SCSIZ

Name	Mode	Reset Value	Description
IVECT	RW	0	Vector to use when signaling an interrupt using MSI or MSI-X. If fewer vectors were allocated by the host, this value is ignored and vector 0 is used.
I	RW	0	Signal command completions using an MSI or MSI-X interrupt. Otherwise, the host polls the command buffers, looking for commands with C and V registers set and N register cleared.
NEWCMDS	WO	0	When written by the host, indicates the number of new commands added in the command buffers since the last time this register was written. Writing 00h has no effect. Always reads back as 00h.
ACKCMDS	RO	0	Number of commands currently in the command buffers that have been acknowledged by the function and are currently under processing. A value of 00h indicates all commands are completed.
CPLCMDS	RO	0	Running count of total number of completed commands. Rolls over back to 0000h after reaching FFFFh.

Sideband Command Set ID

Sideband Command Set ID (SCSI) field is used to identify the commands supported by the attached endpoint model. Following are the list of commands supported by a BSA test function which has *SCSI = 01b* bit set,

```
Ping
Completion Status (UR/CRS/CA)
Hold response to create timeout on Root Port
MSI Interrupt
Upstream Read
Random Upstream Read
Upstream Write
```

```
Random Upstream Write  
Request with different AT bits  
PME/INTx/Error Message  
Set PASID  
Send Data with Poison Bit Set  
Set Ordering  
Set NoSnoop  
Set PASID  
Set TPH  
Start Recording  
Cancel
```

Following are the list of commands supported by a BSA test function which has *SCSI* = *10b* bit set,

```
Ping  
MSI interrupt  
Send Data with Poison Bit Set  
Set Ordering  
Set NoSnoop  
Set PASID  
Set TPH  
Start Recording  
Cancel
```

Currently, the predefined BSA Test Function supports both command sets.

Sideband Commands For Controlling Response and Requests

Sideband command with various opcode and arguments are used to control the response and upstream requests from the BSA test function.

This section covers the following commands:

- [Ping Sideband Command Processor](#)
- [Completion Status Override \(UR/CRS/CA\)](#)
- [Hold Response to Create Timeout on Root Port](#)
- [MSI/X Interrupt](#)
- [Continuous MSI Interrupts](#)
- [Upstream Read](#)
- [Upstream Random Read](#)
- [Upstream Write](#)
- [Upstream Random Write](#)

- [Random Upstream Atomic Operation](#)
- [Request With Different AT Bits](#)
- [Error/PME/INTx Message](#)
- [Send Data With Poison Bit Set](#)
- [Set PASID](#)
- [Set No Snoop](#)
- [Set Traffic Class](#)
- [Set Ordering](#)
- [Set TLP Processing Hint](#)
- [Start Recorder](#)
- [Cancel](#)

Ping Sideband Command Processor

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
0				CID												00h				STATUS					C	N	V	00h				
Value to Invert																																04h
Version Info (0xYYYYMMVV)																																08h
Commit ID																																0Ch

The command with opcode 00h, when completed, replaces 'Value to Invert' with its bitwise-inverted value, 'Version Info' with the version of the BSA software release, and 'Commit ID' with the commit ID that was used to compile the software release.

Completion Status Override (UR/CRS/CA)

31	30	29		28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
0				CID												01h				STATUS					C	N	V	00h					
Reserved																														Code		04h	

Sideband command with opcode 01h configures the model to respond with user-configured Completion Status code for the very next configuration / memory-read request, except for memory-read requests to the Sideband Command region.

Following are the list of valid Completion Status codes:

- 001b - Unsupported Request (UR)
- 010b - Configuration Request Retry Status (CRS)
- 100b - Completer Abort (CA)

Hold Response to Create Timeout on Root Port

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
0				CID												02h				STATUS					C	N	V	00h				
Period in microseconds																																04h

Sideband command with opcode 02h creates the timeout by holding the response for the period mentioned in DWORD[0] for the very next command, except for memory-read requests to the Sideband Command region. The time value is interpreted as scaled PCIe clock time, that is, the apparent real-time the DUT would see in the real world.

MSI/X Interrupt

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
		0		CID												03h				STATUS					C	N	V	00h				
		Reserved																						IVECT				04h				

Sideband command with opcode 03h generates an MSI or MSI-X interrupt with the configured Address and data values specified by the vector 'IVECT'. Note that a the 'Upstream Write' command can be used to generate an equivalent interrupt request.

Continuous MSI Interrupts

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
WEIGHT				CID												04h				STATUS					C	N	V	00h				
COUNT								Reserved								ASTRIDE				64	DSTRIDE				32				04h			
MSI Start Address Lower 32-bit																																08h
MSI End Address Lower32-bit																																0Ch
MSI Address Upper32-bit																																10h
MSI Start Data																																14h
MSI End Data																																18h

Sideband command with opcode 04h generates sequences of COUNT Message Signaled Interrupts. If COUNT is zero, the command runs continuously until explicitly canceled.

The first MSI is sent to the starting address and data value. The next MSI is sent to the same address but with a data value incremented by $2 \times \text{DSTRIDE}$ (incremented by 0 if DSTRIDE is 0). If the next data value is greater than the 'MSI End Data' value, the address is incremented by $2 \times \text{ASTRIDE}$ (incremented by 0 if ASTRIDE is equal to 0) and

the data value reset to the 'MSI Start Data'. After the maximum MSI address and data values are reached, both are reset to their respective starting value.

If '64' is set, then the MSI upper 32-bit value is used. If '64' is cleared, only the lower 32-bits of the MSI address is used. If '32' is set, the full MSI data value is used. If '32' is cleared, only least significant 16 bits of the MSI data value are used.

ASTRIDE must not be equal to 1. DSTRIDE must not be equal to 1. Start values must be less than or equal to end values.

Upstream Read

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
WEIGHT				CID												05h				STATUS					C	N	V	00h				
DWLEN																COUNT				DFCT		Reserved				A	I	04h				
Target Address Lower bits 31..2																												AT		08h		
Target Address Upper 32-bit																																0Ch
Data Buffer offset																																10h

Sideband command with opcode 05h generates read request with up-to the number of DWORDS specified by DWLEN (when A=0x1) or number of bytes specified by DWLEN (when A=0x0) and Target Address pointed by DWORD[2] and DWORD[3], and store the data at the specified offset in the sideband data buffer. If the specified Data Buffer offset is 0xFFFFFFFF, the data read is ignored and the command is considered completed as soon as the request is posted. The request repeats COUNT times. If the 'I' bit is set, the Target Address increments by DWLEN*4 for each request, otherwise the same address is used for all requests.

When emulating Address Translation Requests, A must be equal to 0x1, AT must be equal to 1 and DWLEN must be equal to 2 to request a single address translation. Greater even DWLEN values may be specified to request multiple translations. A non-zero AT value in this command takes precedence over a pending "Request with different AT bits" command. AT filed is used for lower two bits (b1-b0) of Target Address Lower 32bits when A=0x0.

The actual length of each transfer is determined by DWLEN and the distribution function DFCT. If DFCT is 0x0, the "NULL" distribution function is used and all transfers are exactly DWLEN. See Section [Distribution Functions](#) for a list of available distribution functions and how to define user-defined functions.

If COUNT is specified as 0, the function continuously generates the specified read requests, ignoring any data received, until explicitly stopped by a CANCEL command The generated address is incremented up to 255*DWLEN*4, then cycles back to the target address.

Upstream Random Read

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
WEIGHT				CID														06h				STATUS					C	N	V	00h		
COUNT								Rsvd		SPCSZ				STRIDE				REGSZ				Reserved				XFRSZ				04h		
Memory Space Base Lower bits 31..2																												AT				08h
Memory Space Base Upper 32-bit																																0Ch
Rsvd																																10h
SEED																		RO		XFRSZBIN										14h		

Sideband command with opcode 06h generates sequences of read requests of size 2^{XFRSZ} (or 1536 if XFRSZ is 0) that sequentially read the entire region of size 2^{REGSZ} located at a random offset within a memory space of size 2^{SPCSZ} , based at the specified base address. After the region is fully read, a new target region is randomly selected, until COUNT regions have been read. If COUNT is equal to 0, the command runs continuously until explicitly canceled. All data read is ignored. All addresses are respectively aligned.

The starting address stride between two consecutive transfers is $2^{(\text{XFRSZ}+\text{STRIDE})}$ bytes. If STRIDE is equal to 0, the entire region is read without any gaps (except if XFRSZ is equal to 0, in which case the stride is 2048 bytes). The starting address of each subsequent transfers are similarly incremented until the entire region address space is exhausted. If $\text{XFRSZ}+\text{STRIDE}$ is greater than or equal to REGSZ, only one transfer is executed in a region.

When emulating Address Translation Requests, AT must be equal to 1 and XFRSZ must be equal to 3 to request a single address translation. Greater XFRSZ values may be specified to request multiple translations. $\text{XFRSZ} + \text{STRIDE}$ should be greater than or equal to the STU. A non-zero AT value in this command takes precedence over a pending "Request with different AT bits" command.

XFRSZ must be 0, or a value between 2 (4 byte) and 12 (4096 bytes) inclusively. REGSZ must be a value between XFRSZ and 32 (4GB) inclusively. SPCSZ must be a value between REGSZ and 34 (16GB) inclusively. AT must not be equal to 3.

XFRSZBIN is used to specify the random packet lengths to be used for each transfer. XFRSZBIN is effective only when XFRSZ is 0xF, XFRXT is 0x0 and REGSZ is 0x15 (32 kB) and above. Minimum supported XFRSZBIN is 4 Bytes. Standard uniform distribution function is used to select the random payload size from XFRSZBIN.

For example, random payload sizes of 64B, 128B, 256B and 512B can be configured by setting $\text{XFRSZBIN} = 960$ (addition of all the payload sizes $64+128+256+512$ which is same as $64|128|256|512$).

SEED is used to initialize the Standard uniform distribution function. Setting RO as true enables the Relax Ordering bit in the TLP header. Enable Relaxed Ordering bit in the Device Control register need not be set.

Upstream Write

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset																				
WEIGHT								CID								07h								STATUS			C	N	V	00h																						
DWLEN																COUNT								DFCT		Reserved		A	I	04h																						
Target Address Lower bits 31..2																														T	D	08h																				
Target Address Upper 32-bit																																0Ch																				
Data Buffer offset																																10h																				

Sideband command with opcode 07h generates write request with up-to the number of DWORD specified by DWLEN (when A=0x1) or number of bytes specified by DWLEN (when A=0x0), and with Data from specified in Data Buffer offset and Target Address pointed by DWORD[2] and DWORD[3]. If the data buffer offset is 0xFFFFFFFF, unspecified data values are written. The request is repeated COUNT times. If the 'I' bit is set, the Target Address increments by DWLEN*4 for each request, otherwise the same address is used for all requests.

The actual length of each transfer is determined by DWLEN and the distribution function DFCT. If DFCT is 0x0, the "NULL" distribution function is used and all transfers are exactly DWLEN. See [Distribution Functions](#) for a list of available distribution functions and how to define user-defined functions.

If COUNT is specified as 0, the function continuously generates the specified write requests until explicitly stopped by a CANCEL command. The generated address is incremented up to 255*DWLEN*, then cycles back to the target address.

If T is set and A is set, the address is marked as being a translated address (AT = 2).

If D is set and A is set, Deferrable Memory Write requests are issued. No size restrictions are enforced to ensure the DMWr meets minimum update granularity or maximum routability requirements. It is up to the user to ensure valid DMWr requests are specified.

T and D fields are used for lower two bits (b1-b0) of Target Address Lower 32bits when A=0x0.

Upstream Random Write

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
WEIGHT				CID														08h				STATUS					C	N	V	00h		
COUNT								XFRXT		SPCSZ				STRIDE				REGSZ				DATSZ				XFRSZ				04h		
Memory Space Base Lower bits 31..2																													T	D	08h	
Memory Space Base Upper 32-bit																														0Ch		
Data Buffer offset																														10h		
SEED																	RO	XFRSZBIN											14h			

Sideband command with opcode 08h generates sequences of write requests of size 2^{XFRSZ} if XFRXT is 0, or $2^{\text{XFRSZ}} + 2^{\text{(XFRSZ-XFRXT)}}$ if XFRXT is non-zero, or 1536 if XFRSZ is 0, that sequentially write the entire region of size 2^{REGSZ} located at a random offset within a memory space of size 2^{SPCSZ} , based at the specified base address. After the region is fully written, a new target region is randomly selected, until COUNT regions have been written. If COUNT is equal to 0, the command runs continuously until explicitly canceled. All addresses are respectively aligned.

The starting address stride between two consecutive transfers is $2^{\text{(XFRSZ+STRIDE)}}$ bytes. If STRIDE is equal to 0, the entire region is written without any gaps (except if XFRSZ is equal to 0, in which case the stride is 2048 bytes). The starting address of each subsequent transfers are similarly incremented until the entire region address space is exhausted. If XFRSZ+STRIDE is greater than or equal to REGSZ, only one transfer is executed in a region.

The data payload of the write requests is read from the specified Data Buffer offset, up to 2^{DATSZ} bytes long. The content of the Data Buffer is used as many times as necessary to fill the write request should it be longer than 2^{DATSZ} bytes. If DATSZ is equal to 0, the Data Buffer offset value is ignored and all written data is random/unspecified.

XFRSZ must be 0, or a value between 2 (4 bytes) and 12 (4096 bytes) inclusively. XFRXT must be less than or equal to XFRSZ-2. REGSZ must be a value between XFRSZ and 32 (4GB) inclusively. SPCSI must be a value between REGSZ and 34 (16GB) inclusively.

For example, with XFRSZ = 5, XFRXT = 1, STRIDE = 2, the command would write 48-bytes ($2^5 + 2^{(5-1)} = 32 + 16 = 48$) at addresses 128-bytes apart ($2^{(5+2)} = 128$).

If T is set, the address is marked as being a translated address (AT = 2).

If D is set, Deferrable Memory Write requests are issued. No size restrictions are enforced to ensure the DMWr meets minimum update granularity or maximum routability requirements. It is up to the user to ensure valid DMWr requests are specified.

XFRSZBIN is used to specify the random packet lengths to be used for each transfer. XFRSZBIN is effective only when XFRSZ is 0xF, XFRXT is 0x0 and REGSZ is 0x15 (32

kB) and above. Minimum supported XFERSZBIN is 4 Bytes. Standard uniform distribution function is used to select the random payload size from XFERSZBIN.

For example, random payload sizes of 64B, 128B, 256B and 512B can be configured by setting XFERSZBIN = 960 (addition of all the payload sizes 64+128+256+512 which is same as 64|128|256|512).

SEED is used to initialize the Standard uniform distribution function. Setting RO as true enables the Relax Ordering bit in the TLP header. Enable Relaxed Ordering bit in the Device Control register need not be set.

Random Upstream Atomic Operation

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
WEIGHT				CID												09h				STATUS					C	N	V	00h				
COUNT								Rsvd		SPCSZ				I		STRIDE					128	64	32		CS	SW	AD	04h				
Memory Space Base Address Lower 32-bit																													T	0	08h	
Memory Space Base Address Upper 32-bit																															0Ch	
Data Buffer offset																															10h	

Sideband command with opcode 09h generates atomic operation with the argument from the specified offset in the sideband data buffer and the response replacing the argument data at that same offset. If the specified Data Buffer offset is 0xFFFFFFFF, an unspecified argument value is used, the response is ignored and the command is considered completed as soon as the request is posted. The request is repeated COUNT times.

If SPCSZ and STRIDE are 0, the specified address is used repeatedly as the target for the atomic operation. The target address must be aligned with the largest enabled operation size.

If “I” is 1, the address specifies the base address of (and must be aligned with) a 2^{SPCSZ} region. The Target Address is incremented by 2^{STRIDE} (which must be aligned with the largest enabled operation size) after each generated operation and loops around to the base address when the end of the region is reached. STRIDE must be less than SPCSZ.

If “I” is 0, the address specifies the base address of (and must be aligned with) a 2^{SPCSZ} region. The Target Address is randomly selected within the region and aligned to 2^{STRIDE} (which must be aligned with the largest enabled operation size). STRIDE must be less than SPCSZ.

If T is set, the address is marked as being a translated address (AT = 2).

The actual operation and length of each transfer is randomly determined based on the CS, SW, AD, ‘128’, ‘64’, and ‘32’ bits that are set. Only operation and size corresponding to set bits are in the pool of selectable operation. At least one CS, SW, or AD bit must be set. At

least one '32', '64', or '128' bit must be set. The '128' bit can only be set if the CS bit is also set.

If COUNT is specified as 0, the function will continuously generate the specified atomic requests, ignoring any data received, until explicitly stopped by a CANCEL command.

Request With Different AT Bits

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
0				CID												0Ah								STATUS					C	N	V	00h
Reserved																												AT			04h	

Sideband command with opcode 0Ah configures the model to issue the next upstream read or write request with AT bits specified in the 'AT' field. The ATS capability is present in functions that support this sideband command, but the Enable bit in the ATS Control Register is ignored and the AT bits are set accordingly whether or not the Enable bit is set.

This command is applied to the next upstream command only if the AT or T field specified in that command is equal to 0. Otherwise, it remains pending and is applied to the next applicable upstream command.

Error/PME/INTx Message

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset						
CID																0Bh								STATUS					C	N	V	00h						
Reserved																				Err Index						Code												04h

Valid and supported Message Code:

- 14h - PM_Active_State_Nak
- 18h - PM_PME
- 1Bh - PME_TO_ACK
- 20h - Assert_INTA
- 21h - Assert_INTB
- 22h - Assert_INTC
- 23h - Assert_INTD
- 24h - Deassert_INTA
- 25h - Deassert_INTB

- *26h - Deassert_INTC*
- *27h - Deassert_INTD*
- *30h - Generate Correctable Error message as per Err Index*
- *31h - Generate Uncorrectable Error message as per Err Index*

The following table shows supported Err Index to generate Correctable AER error messages:

<i>Err Index</i>	<i>Correctable Error</i>
0	Correctable Receiver Error
6	Correctable Bad TLP
7	Correctable Bad DLLP
8	Correctable Replay Num Rollover
12	Correctable Replay Timer Timeout
13	Correctable Advisory Non-Fatal Error
14	Correctable Internal Error
15	Correctable Header Log OverFlow

The following table shows supported Err Index to generate Uncorrectable AER error messages:

<i>Err Index</i>	<i>Correctable Error</i>
4	Uncorrectable Data Link Error
5	Uncorrectable Surprise Down Error
12	Uncorrectable Poisoned TLP Received
13	Uncorrectable Flow Control Error
14	Uncorrectable Completion Timeout
15	Uncorrectable Completer Abort
16	Uncorrectable Unexpected Completion
17	Uncorrectable Receiver Overflow

18	Uncorrectable Malformed TLP
19	Uncorrectable ECRC Error
20	Uncorrectable Unsupported Request
21	Uncorrectable ACS Violation
22	Uncorrectable Internal Error
23	Uncorrectable MultiCast Blocked TLP
24	Uncorrectable Atomic Op Egress Blocked
25	Uncorrectable TLP Prefix Blocked Egress
26	Uncorrectable Poisoned TLP Egress Blocked

Programming sideband command with opcode 0Bh and with valid message code, Model sends PME, INTx, or an error message to the Root Complex.

Send Data With Poison Bit Set

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
0				CID												0Dh							STATUS					C	N	V	00h	

Sideband command with opcode 0Dh configures the model to respond with poison bit set for the next memory read command, except for memory-read requests to the Sideband Command region.

Set PASID

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset				
0				CID												12h							STATUS					C	N	V	00h					
PASID																																				04h

Sideband command with opcode 12h configures the test function to use the specified PASID until the command is canceled using a Cancel command.

Note that this command does not require that the test function advertise a PASID Capability.

Set No Snoop

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
0				CID												14h							STATUS					C	N	V	00h	

Sideband command with opcode 14h configures the model to set No Snoop bit for all memory request commands issued by the function, until the command is canceled by using a `Cancel` command.

Set Traffic Class

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
0				CID												15h				STATUS					C	N	V	00h				
Reserved																				TC				04h								

Sideband command with opcode 15h configures the model to set the Traffic Class to the specified value for all memory request commands issued by the function, until the command is canceled by using a `Cancel` command. If this command is not used, all memory requests are assigned Traffic Class 0.

Set Ordering

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
0				CID												16h				STATUS					C	N	V	00h				
Reserved																												I	R	04h		

Sideband command with opcode 16h sets the Relaxed (if R is set) and/or ID-Based (if I is set) Ordering bit on all outgoing upstream memory and message requests until it is canceled using a `Cancel` command. The IDO Request Enable and IDO Completion Enable bits in the Device Control 2 register need not be set.

Set TLP Processing Hint

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
0				CID												17h				STATUS					C	N	V	00h				
Reserved																												PH		04h		

Sideband command with opcode 17h sets the TH bit and assign the PH value on all outgoing upstream memory requests until it is canceled using a `Cancel` command. When this command is not used, the TH bit is always cleared and PH is set to 0b00.

Start Recorder

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
0				CID												1Ch				STATUS					C	N	V	00h				
NumTLPs																SDSIZ								WT	TLPSIZ				04h			
Trace Buffer Offset in Sideband Data																												08h				

Sideband command with opcode 1Ch starts the trace mechanism to record all the in-band TLPs received by the root device in the trace buffer located at the specified offset in the Sideband Data region. This command is only valid in all functions and apply to all TLPs received by the root device regardless of the function's location in the topology.

<i>Name</i>	<i>Description</i>
NumTLPs	Maximum number of TLPs to record.
SDSIZ	Number of bytes available in the Trace Buffer, specified as $2^{\wedge}\text{SDSIZ}$. Minimum value is 5. If the specified Trace Buffer goes beyond the Sideband Data memory, it is truncated accordingly.
WT	Wait for Trigger signal from RC when this bit is set to 1b'1. Config write to ClassCode and Revision configuration register at address 08h with value <code>0x16c30800</code> is used as trigger signal to start recording the TLP's. Config write to ClassCode and Revision configuration register at address 08h with value <code>0x16c30801</code> is used as trigger signal to stop recording the TLP's.
TLPSIZ	Maximum number of DWORDs to record for a single TLP, specified as $2^{\wedge}\text{TLPSIZ}$.
STATUS	In addition to the standard completion status code, this command indicates successful completion with one of the command-specific status codes s follows: 0x1 - Recording reached end of buffer 0x2 - Recording was stopped by user 0x3 - Recording reached NumTLPs 0x4 - Recorder command is processed 0x6 - Recording was stopped for an unspecified reason 0x7 - A recording is already active

There can only be one recording command active in the entire topology at any given time. If a new "Start Recorder" command is found in the Sideband Command queue, it is going to be immediately acknowledged and marked as completed with a 0x7 STATUS. If a subsequent recording command uses a trace buffer that overlaps the trace buffer of a previous recording command, the new TLP recording overwrites the previous recorded ones. Recording automatically stops when the specified number of TLPs has been recorded, or the trace buffer overflows.

The TLP trace buffer is filled with a sequence of the following structure:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Buffer Offset																								
Reserved								CID								CTL																00h																								
																																04h																								
TLP Data Bytes 0..3																																08h																								
TLP Data Bytes 4..7																																0Ch																								
...																																...																								
TLP Data[Length-1]																																CTL X 4																								

Name	Description
CID	“Start Recording” Command ID. Set by the tracing mechanism to the CID specified in the “Start Recording” Command the trace belongs to. A CID of 0x0000 indicates the end of the recording.
CTL	Command Trace Length. Specifies the length of recorded TLP in DWORDS, including the DWORD containing the CID and CTL. The next TLP record can be found at <current TLP offset> + (CTL x 04). A value of 0 indicates the end of the recording.
Timestamp	Indicates the relative time the TLP was received and recorded. Units are unspecified and roll over on overflow.
TLP Data	Records the TLP data (including any prefix) as received. Byte 0 (Fmt/Type) is located in bits 31:24.

Cancel

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Offset
0								CID								1Dh								STATUS								00h

Sideband command with opcode 1Dh cancels all pending or currently executing commands with the specified CID (or all commands if CID is 0x0000). If a command with the specified CID was not completed when it was canceled, its completion status is set to 0x2. The `Cancel` command always completes successfully, even if no commands were actually canceled.

Independent Sideband Access

In addition to normal accesses as PCIe transaction through the host's PCIe Root Complex, all configuration and memory resources of all PCIe devices and functions in the PCIe topology are reachable directly from a 4KB window in the system address space. This allows access to the entire PCIe topology without requiring that the host's PCIe Root Controller have completed PCIe linkup, enumerated the PCIe topology, assigned base

addresses to the various BAR registers, and configured address apertures in the various bridges.

The base address of the 4KB address window is specified as a parameter to an instance of the AXI4 Responder Transactor in HDL. The bus protocol for this physical space access is AXI4 (64-bit Data) and a corresponding initiator port in the RTL design must be provided and reserved for all access in that 4KB address window directly through decoding of the system address space. That AXI4 initiator port must be directly connected to the BSA sideband responder port in the simulation or emulation environment.

<i>Offset in 4KB Window</i>	<i>End</i>	<i>Symbol</i>	<i>Name</i>
00h	03h	SBDF	Target Device Bus, Device, Function address00h: Bits 2:0 Function01h: Bits 12:8 Device02h: Bits 23:16 Bus03h: Bits 31:24 Reserved
04h	07h	SADDR	Indirect Address (Offset into Config or BAR MMIO spaces)
08h	0Bh	SCFG	Config Read/Write
10h	13fh	SBAR0	BAR0 Read/Write
14h	17h	SBAR1	BAR1 Read/Write
18h	1Bh	SBAR2	BAR2 Read/Write
1Ch	1Fh	SBAR3	BAR3 Read/Write
20h	23h	SBAR4	BAR4 Read/Write
24h	27h	SBAR5	BAR5 Read/Write

The access to any PCIe function in the topology is through the indirect addressing registers SBDF and SADDR.

For example, to perform a config read of function 1:02.3 at offset address 0x10:

```
Set SBDF = 0x00010203 (one byte each for B-D-F)
Set SADDR = 0x00000010
Read data from SCFG
```

Read transaction is routed to the configuration space of the required PCIe function instance at the required offset and data returned on AMBA bus

For example, to write to MMIO BAR4 of function 2:01.3 at offset 0xFFED:

```
Set SBDF = 0x00020103 (one byte each for B-D-F )
Set SADDR = 0x0000FFED
Write to SBAR4
```

Write transaction is routed to BAR4 MMIO space of required PCIe function and the data on the AMBA bus is written to the MMIO space at the required offset

Note that the sideband channel performs a pre-enumeration of the PCI topology, assigning bus numbers starting with 1. A map of the accessible PCIe functions is generated at runtime and display on the simulation or emulation output. When the host performs its normal enumeration process, it may assign different bus numbers. However, the sideband channel continues to use its original pre-enumeration bus numbers.

6.5

Setting the Priority of Upstream Read Requests Over Write Requests

By default, upstream memory read and write requests, as well as completions to downstream requests, are sent in the order in which they are submitted by the PCIe topology. If a different traffic profile is required to achieve specific bandwidth utilization goals, it is possible to prioritize different types of upstream traffic over others.

For all prioritization schemes, a request is only considered for prioritization if there are sufficient credits for it to be sent.

Note that upstream prioritization is done strictly based on the corresponding type but still respects the PCIe ordering rules (that is, a completion cannot pass a previously submitted write request and a read request cannot pass a previously submitted completion).

Three upstream prioritization modes are supported as follows:

<i>Value</i>	<i>Priority Algorithm</i>
NONE	Prioritization is disabled. TLPs are sent in the order they were submitted. This is the default.
ROUND_ROBIN	Plain round-robin priority between completions, read, and write requests, regardless of the number of respective pending requests.

Value	Priority Algorithm
MODIFIED_RR	Read requests have the highest priority, followed by completions and writes. If the number of pending write requests exceeds the high watermark, then priority becomes round robin between writes and reads, with completions having the lowest priority. Priority reverts to lowest level for writes when the number of pending write requests decreases below the low watermark. Similarly, if the number of pending completions exceeds the high watermark, then priority becomes round robin between reads and completions with later-posted writes having the lowest priority. Priority reverts to reads when the number of pending completions decreases below the low watermark.
CRW_WRC	Completions have the highest priority, followed by reads, followed by write. If the number of pending write requests exceeds the high watermark, then writes have higher priority than reads and completions. Priority reverts to lowest level for writes when the number of pending write requests decreases below the low watermark. Similarly, if the number of pending reads exceeds the high watermark, then reads have higher priority than completions. Priority reverts to completions when the number of pending read requests decreases below the low watermark.
FREQ_COUNT	The high watermark value for each queue specifies a relative expected frequency count for each request type. The request with the largest difference between its current frequency count and its expected frequency count has the highest priority. The low watermark is ignored.

The `<virtDev:UpPriority>` XML element inside the `<virtDev:Port>` element is used to enable and configure upstream prioritization. The values shown in the following example are default values.

```
<virtDev:Port>
  <virtDev:Name>link0</virtDev:Name>
  <virtDev:HdlPath>wrapper.pcie_driver_ep</virtDev:HdlPath>
  <virtDev:DebugLevel>0</virtDev:DebugLevel>
  <virtDev:Type>DPI</virtDev:Type>

  <virtDev:CycleAccurate>>false</virtDev:CycleAccurate>
  <virtDev:MaxPayloadBytes>4096</virtDev:MaxPayloadBytes>
  <virtDev:MaxLinkSpeed>0</virtDev:MaxLinkSpeed>
  <virtDev:UpPriority>
    <virtDev:Mode>NONE</virtDev:Mode>
    <virtDev:WriteQueue>
      <virtDev:HighWatermark>128</virtDev:HighWatermark>
      <virtDev:LowWatermark>16</virtDev:LowWatermark>
      <virtDev:Max>512</virtDev:Max>
    </virtDev:WriteQueue>
  </virtDev:UpPriority>
</virtDev:Port>
```

```
<virtDev:ReadQueue>
  <virtDev:HighWatermark>128</virtDev:HighWatermark>
  <virtDev:LowWatermark>16</virtDev:LowWatermark>
  <virtDev:Max>512</virtDev:Max>
</virtDev:ReadQueue>
<virtDev:CompletionQueue>
  <virtDev:HighWatermark>128</virtDev:HighWatermark>
  <virtDev:LowWatermark>16</virtDev:LowWatermark>
  <virtDev:Max>512</virtDev:Max>
</virtDev:CompletionQueue>
</virtDev:UpPriority>
</virtDev:Port>
```

The `<virtDev:Mode>` element specifies the prioritization algorithm, using one of `NONE`, `ROUND_ROBIN`, `MODIFIED_RR`, `CRW_WRC`, or `FREQ_COUNT`. Default is `NONE`.

The `<virtDev:WriteQueue>`, `<virtDev:ReadQueue>`, and `<virtDev:CompletionQueue>` specify the maximum size, high watermark, and low watermark for the pending write requests, read requests, and completion requests respectively. Optional. Default to 512, 128, and 16 respectively.

The maximum high-watermark of a request queue is 255. The maximum low-watermark of a request queue is 254 and must be lower than the configured high-watermark.

The maximum size of a request queue is 65535. If a pending request queue reaches its maximum specified value, the `SNPS::P`.

ACS Testing

By default, if the target address of an upstream request is located in another function in the PCIe topology, the upstream request is routed directly to the peer target function at the earliest opportunity. To re-direct all peer-to-peer requests to the Root Complex, where it will be up to the Root Complex to correctly route the request and follow-up response, peer-to-peer re-direction must be enabled (equivalent to setting the ACS P2P Request Redirection Enable (R) and ACS P2P Completion Redirection Enable (C) bits in all ACS Control Registers) using the following static function: `SNPS::PCIe::Bridge::enablePeerToPeerRedirection(true);`

Address Invalidation Testing

All Address Invalidation messages are immediately responded to. There is no support for coalescing invalidation tags in a single response. By default, only one response is sent (`CC = 1`) with no delays. The BSA function may be configured to send multiple responses

for each invalidation request or to delay each response by a specified number of PCIe clock cycles using the following XML elements (showing their default value):

```
<virtDev:Function>
  <virtDev:Name>BSA</virtDev:Name>
  ...
  <virtDev:ATS>
    <virtDev:CC>1</virtDev:CC>
    <virtDev:RespDelay>0</virtDev:RespDelay>
  </virtDev:ATS>
</virtDev:Function>
```


5

Appendix: Supported Software Drivers

There is a complex discovery and configuration process involved in getting devices ready to be used, as well as submitting BSA test commands. The BSA-VDM solution includes convenience software driver components to ease the implementation and execution of BSA compliance tests.

The C++ header files defining these predefined software driver components can be found in the `$SNPS_VIRTDEV_HOME/drivers/include` directory.

The precompiled object files for the x86 and ARM64 architecture can be found in the `$SNPS_VIRTDEV_HOME/drivers/libABI?/x86` and `$SNPS_VIRTDEV_HOME/drivers/libABI?/aarch64` directories respectively.

Detailed documentation of the various APIs can be found in Doxygen output.

Sideband Channel Software Driver

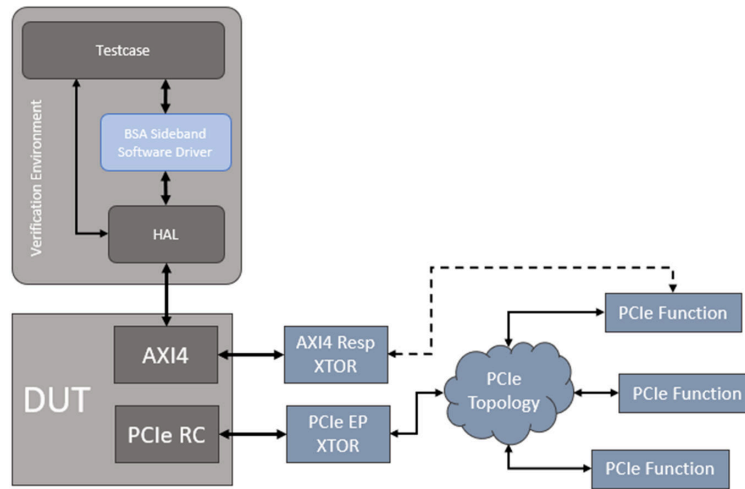
The Sideband channel software driver provides convenience APIs to perform configuration and memory read/write transaction on a PCIe function via the sideband channel. It also provides a convenience C++ struct declaration for sideband command buffers that abstract the location and layout of various command structures and parameters. A single instance of the `SNPS::SW::BSA::Driver` per host is sufficient.

See `SNPS::SW::BSA::Driver` class for more details.

Hardware Abstraction Layer

The BSA sideband channel software driver performs its task by read and write transactions via the sideband AXI4 initiator accessible to the user-implemented 4KB address window. As the means of doing so differs from verification environment to verification environment and from host model to host model, those operations are performed through a Hardware Abstraction Layer (HAL) that must be implemented by the user

Figure 8 Hardware Abstraction Layer



```
#include <BSADriver.hh>

SNPS::Utils::Logger    log("test");
SNPS::SW::BSA::Driver  sbDrvr(log);
...
int main(...)
{
    ...
    uint32_t IDs;
    sbDrvr.cfgRd(1, 0, 0, 0x000, 4, IDs);
    ...
}
```

The entire HAL that must be provided is contained within the `SNPS::SW::BSA::Driver::HAL` naming scope. See TBD (Doxygen) for more details.

Every HAL API required by the BSA software driver is specified as a static method, but no implementation is supplied. It must be supplied by the user and can be implemented as a C++ global function in the verification environment.

```
#include <BSADriver.hh>

SNPS::PCIE::Status_t SNPS::SW::BSA::Driver::HAL::read(void* context,
    uint16_t addr, uint32_t &rdat)
{
    uint64_t physAddr = 0x00000000000000FF0000UL + addr;
    rdat = *((uint32_t*) physAddr);
    return SNPS::PCIE::SUCCESS;
}
```

An optional user-defined opaque context pointer, specified when instantiating the `SNPS::SW::BSA::Driver` class, is passed to every HAL function called by that instance.

This provides a mechanism for distinguishing which instance of the BSA software driver is using the HAL, should multiple instances exist in a verification environment.